More on time in low space, after OpenAI

Ryan O'Donnell

Cook–Mertz clean composition lets recursively computed values share the same physical registers. We apply that lemma to circuits whose gates are computed by short blackboard conversations. The resulting evaluation theorem separates the amount of data locally available to a party from the communication between parties, and gives the following simulations of deterministic time T.

Machine model	**Work space**
A fixed number of one-dimensional writable tapes	$\widetilde{O}(T^{1/2})$
One one-dimensional writable tape	$\widetilde{O}(T^{2/5})$
One fixed d-dimensional writable tape	$\widetilde{O}(T^{2d/(3d+2)})$
A RAM with $O(\log T)$-bit words and addresses	$\widetilde{O}(T^{2/3})$

The algebraic ingredient is the clean-composition lemma of Cook and Mertz [CM24], in the polynomial form stated by Sibgatullin and Williams [SW26, Lemma 2.5]. We use this lemma rather than reprove the tree-evaluation algorithm. The additional argument places public inputs outside the enumeration of conversations, yielding an input-sensitive space bound.

Two memory reductions supply the applications. A computation making R visits to constantly many B-bit blocks per visit uses $\widetilde{O}(B + R)$ space; this is a special case of the stronger external-memory theorem of Sibgatullin and Williams [SW26], reproved here for completeness. When each visit touches one block, short conversations improve the bound to $\widetilde{O}(B + R^{2/3})$. This develops the one-tape construction of [OAI26]; the first application recovers Williams's multitape exponent [Wil25]. We also give the version in which a joint visit communicates κ bits, obtaining $\widetilde{O}(B + (R\kappa)^{2/3})$.

All machines are fixed and deterministic. Each writable tape has one nearest-neighbor head; a fixed number of read-only input heads is allowed. The simulator receives a binary cap T at least the input length and returns the finite-control state and halt/timeout outcome. It has read-only access to the input, unrestricted running time, and need not materialize the final memory. Tildes suppress polylogarithmic factors; integer scales are rounded as needed.

1 Blackboard circuits

A *width-w blackboard circuit* is an acyclic circuit whose wires carry bitstrings of length at most w. At a gate v, partition the incoming wire occurrences into ordered groups

$$H_v,\ A_{v,1}, \ldots, A_{v,m_v},$$

each containing at most w bits in total. A group can consist of one word or many individual bits. Concatenation gives its input string.

Initially, H_v is written on a public blackboard, and party i privately receives $A_{v,i}$. A supplied deterministic protocol appends at most $q_v \le w$ bits. The blackboard determines the next speaker and when the protocol stops; a message bit depends only on the blackboard and the speaker's private input. At termination, a publicly selected party computes the gate's output from its private input and the blackboard. The output has at most w bits and is not charged as communication.

The protocol's answer is the gate value. Sources have supplied values, and the circuit has one designated output. Empty input groups are allowed; a party with empty private input handles a gate that needs only its public input.

Uniformity. Let N bound the circuit size, input occurrences, number of parties, width, and public input length. Here tildes hide powers of $\log N$. Wiring, group boundaries, and protocol parameters are accessible in $\operatorname{polylog} N$ space. Source coordinates and local protocol routines are computable in $\widetilde{O}(w)$ space from explicit Boolean arguments and the original read-only input. The routines are total, including on inconsistent proposed transcripts, and make no calls to the circuit evaluator. The local programs, not exponentially large protocol trees or truth tables, are part of the description.

Direct dependency edges from gates toward their inputs. Let h be the ordinary dependency depth, and define the *path communication*

$$Q(C) = \max_{\pi} \sum_{\substack{e \in \pi:\ e \text{ supplies a private} \\ \text{input to gate } v}} q_v, \tag{1}$$

where π ranges over output-to-source paths. An edge supplying H_v contributes zero. Repeated input occurrences retain their respective public or private roles.

Theorem 1 (Blackboard-circuit evaluation)**.** *A width-w blackboard circuit of depth h and path communication Q can be evaluated deterministically in*

$$\boxed{\widetilde{O}(w + h + Q)}$$

work-space bits. If every gate communicates at most q bits and every dependency path uses at most r private-input edges leaving communicating gates, the bound is $\widetilde{O}(w + h + rq)$.

The natural recursive evaluator retains a public word and partial conversation while obtaining a private input, using $\widetilde{O}(wh + Q)$ space. The theorem removes the multiplication of word storage by depth.

Corollary 2 (Ordinary word circuits)**.** *A depth-h circuit whose gate operations have total input length at most w, output length at most w, and explicit-input space complexity $\widetilde{O}(w)$ is evaluable in $\widetilde{O}(w + h)$ space.*

Proof. Put each gate's inputs on its initial blackboard. No communication is needed, so $Q = 0$. □

There is no constant-fan-in assumption: a public group may contain w one-bit wires. Tree evaluation with arity a, b-bit values, and height h has width ab, recovering the Cook–Mertz bound $\widetilde{O}(ab + h)$. A binary operation on two b-bit words can instead use width b: one input is public and the other belongs to the sole party.

An addressed read is another zero-communication gate:

$$\mathsf{Read}(j; A_1, \ldots, A_m) = A_j.$$

The address j is public, party i holds A_i, and the public address selects the output party.

2 The Cook–Mertz ingredient

A routine *cleanly adds* a fixed vector x to designated registers if its net effect is to translate those registers by x and restore every other register, for every initial assignment to the entire register pool. The initial contents need not be zero. This is the clean-computation notion of Cook and Mertz [CM24].

We use the following form of their composition lemma; see also the degree-explicit statement in [SW26, Lemma 2.5]. A *bank* below is simply a tuple of field registers.

Lemma 3 (Cook–Mertz clean composition)**.** *Let $\mathbb{F}_p$ be a prime field and let $f : \mathbb{F}_p^a \to \mathbb{F}_p$ be a polynomial of degree less than $p - 1$. Suppose a routine cleanly adds $x \in \mathbb{F}_p^a$ to a bank R, and a second routine cleanly subtracts it. Then one can cleanly add $f(x)$ to a separate target register using the same register pool.*

The composition consists of one round for each $s \in \mathbb{F}_p^\times$:

$$R \leftarrow sR; \qquad \textit{add } x; \qquad \textit{target} \leftarrow \textit{target} - f(R); \qquad \textit{subtract } x; \qquad R \leftarrow s^{-1}R. \tag{2}$$

Apart from the supplied routines and local evaluation of f, the round needs only field scalars and coordinate counters.

The associated sampling identity is

$$f(x) = -\sum_{s \in \mathbb{F}_p^\times} f(sJ + x) \qquad (\deg f < p - 1), \tag{3}$$

for an arbitrary initial bank value J. This is the all-nonzero-elements form of the Cook–Mertz interpolation identity.

Two uses of the lemma will matter. First, f may read other registers as parameters: they need only be restored before each sample, not saved separately. Second, the sample increment in (2) may itself be implemented by a routine with the same net effect, restoring all nontarget registers. Substituting such a routine leaves the composition unchanged. For each fixed parameter value, the same sample identity applies. These observations let us nest the lemma while keeping one physical register pool.

The field. All degrees below are at most $\Delta = Nw$. Find a prime $2\Delta < p < 4\Delta$ by trial division; Bertrand's postulate guarantees one. The search and arithmetic modulo p use $O(\log N)$ bits. We use one field element per Boolean coordinate and suppress the resulting logarithmic factor.

Local Boolean functions. For $u \in \{0,1\}^w$, set

$$L_u(Z) = \prod_{j:u_j=1} Z_j \prod_{j:u_j=0} (1 - Z_j).$$

Pad input groups to w bits. The usual multilinear extension of a local function $b : \{0,1\}^w \times \{0,1\}^w \to \{0,1\}$ is

$$\widehat{b}(H, A) = \sum_{u,v \in \{0,1\}^w} b(u, v) L_u(H) L_v(A). \tag{4}$$

It agrees with b on Boolean arguments and has degree at most w in each word argument. Given explicit field vectors, enumerate u, v and accumulate the sum to evaluate (4) in $\widetilde{O}(w)$ space. This lookup uses no recursive calls; its scratch is discarded after obtaining the scalar answer.

3 Blackboard composition

We prove Theorem 1 by applying Lemma 3 first to the public input and then, inside each sample, to one private input at a time.

3.1 A fixed conversation separates the parties

Fix a gate with inputs $H, A_1, \ldots, A_m$. Pad its protocol with zero messages after stopping, so every conversation has exactly q bits. The output rule ignores the padding.

For a proposed conversation $\tau \in \{0,1\}^q$, party i scans the message positions. Whenever the public rule makes it the speaker, it checks the proposed bit using only H, A_i and the proposed prefix. All parties' checks pass exactly for the actual conversation: induction on the message position forces each next bit to be correct.

For output coordinate z, fold the answer into these checks. Define

$$b_{\tau,i,z}(H, A_i) = \begin{cases} 0, & \text{if party } i\text{'s check fails,} \\ \text{its proposed output bit } z, & \text{if it passes and is the output party,} \\ 1, & \text{if it passes and is not the output party.} \end{cases} \tag{5}$$

The output party is determined publicly by H, τ. Thus each $b_{\tau,i,z}$ is a total local Boolean function of two words, with τ as an explicit parameter.

An incorrect conversation contributes zero to the following expression; the actual conversation contributes the output bit. Therefore, over $\mathbb{F}_p$,

$$X_v[z] = \sum_{\tau \in \{0,1\}^{q_v}} \prod_{i=1}^{m_v} b_{v,\tau,i,z}(H_v, A_{v,i}). \tag{6}$$

Extend each factor separately and write

$$F_{v,z}(H; A_1, \dots, A_{m_v}) = \sum_{\tau \in \{0,1\}^{q_v}} \prod_{i=1}^{m_v} \widehat{b}_{v,\tau,i,z}(H, A_i). \tag{7}$$

Its degree in H is at most $m_v w \le \Delta$, and its degree in any one A_i is at most w. Summing over conversations does not increase these degrees. At the actual inputs, $F_{v,z}$ equals $X_v[z]$.

3.2 Compose the clean routines

Use three global banks in $\mathbb{F}_p^w$. Inductively implement

$$\mathsf{Add}(v, \lambda, R, I): \qquad R[I] \leftarrow R[I] + \lambda X_v,$$

restoring every other coordinate, for arbitrary entry contents. Here I is a segment of length $|X_v|$. Sources are immediate. To translate a bank by an input group, invoke these routines on its constituent wires and the corresponding consecutive segments. This requires only a wire index and position counter, not another bank. Write this operation as $\mathsf{Shift}(\text{group}, \lambda, R)$.

For an internal gate, write $R_{\text{out}}[z]$ for coordinate z of the target segment and call the other banks R_H, R_A. Let the latter initially contain J_H, J_A, and let the actual input groups be $h, a_1, \dots, a_m$.

Outer composition: the public input. Use Lemma 3 with clean translation by h into R_H. At sample s, its contents are

$$R_H = sJ_H + h.$$

We must implement the sample increment $-\lambda F_{v,z}(R_H; a_1, \dots, a_m)$ for each output coordinate z. This is done as follows, restoring both input banks.

Inner composition: one private input. Enumerate $\tau \in \{0,1\}^{q_v}$ and multiply its factors in order. For factor i, apply Lemma 3 to the clean translation by a_i into R_A, with polynomial

$$A \longmapsto \widehat{b}_{v,\tau,i,z}(R_H, A).$$

The bank R_H is a parameter, restored by every child call. The inner composition returns the field scalar

$$-\sum_{r \neq 0} \widehat{b}_{v,\tau,i,z}(sJ_H + h, rJ_A + a_i) = \widehat{b}_{v,\tau,i,z}(sJ_H + h, a_i). \tag{8}$$

A scalar accumulator, initially zero, serves as the inner target. Multiply the result into the current product and discard it. After the last factor, subtract λ times the product from $R_{\text{out}}[z]$.

Summing over τ gives precisely the required outer sample increment. The outer clean composition therefore adds

$$\lambda F_{v,z}(h; a_1, \dots, a_m) = \lambda X_v[z]$$

to the target. This proves the inductive contract.

The nesting, including its data lifetimes, is

$$\underbrace{\text{clean composition on } H}_{\text{no conversation across calls to } H} \left[\text{for each } z: \sum_{\tau} \prod_{i} \underbrace{(\text{clean composition on } A_i)}_{\tau \text{ retained across calls to } A_i} \right]. \tag{9}$$

All lookup scratch is released before recursion. Only scalar sums and products survive an inner call. The reverse public translation occurs after the coordinate and conversation loops have ended.

3.3 Space

A child may temporarily borrow a bank holding an ancestor's shifted input or partial output. Its clean contract restores that bank before returning, regardless of its entry contents. Thus no ancestor saves a word vector. Multiple uses of a predecessor cause no difficulty: the identities above concern its fixed value, not independent copies.

The three banks and reusable local scratch use $\widetilde{O}(w)$ bits. On a dependency into H_v, a suspended frame contains only polylog N bits: identifiers, field scalars, interpolation points, and group indices. On a dependency into $A_{v,i}$, it also retains the q_v-bit conversation. Storage for that conversation is released before the reverse public call, so descendants may overwrite it.

Active calls form a dependency path. The short bookkeeping costs $\widetilde{O}(h)$, and retained conversations cost at most $Q(C)$. The current unsuspended conversation has at most w bits. Hence

$$\text{space} = \widetilde{O}(w + h + Q(C)).$$

All loops terminate and all recursive calls go to predecessors. A clean addition to an initially zero target computes the output, completing the proof of Theorem 1.

The public/private distinction is essential to this argument. Expanding the sum over 2^{q_v} conversations into an ordinary binary tree would put depth q_v above *both* kinds of input. The nested use of clean composition charges it only to private dependencies.

4 External memory: read, compute, write

For the rest of the paper, all numerical parameters are polynomially bounded in T, and tildes hide powers of $\log T$. A block computation has m persistent, disjoint blocks of B bits and a controller of $c = O(\log T)$ bits. The controller selects a tuple of at most a distinct blocks, where a is fixed. A deterministic *visit* reads these blocks, updates them, and returns the next controller. Other blocks are unchanged.

Selectors are accessible in polylog T space. Initial block coordinates, the initial controller, and visits on explicit inputs are computable in $\widetilde{O}(B)$ space. All routines are total. We seek the controller after R visits; a short visit counter permits identity padding afterward.

Sibgatullin and Williams [SW26, Theorem 1.1] prove a stronger external-memory theorem: their free-input parallel model with P processors, M internal bits per processor, S external bits, and I I/O rounds is simulable in

$$O\big(M + I \log(PSM)\big)$$

space under their uniformity assumptions. It permits free access to the original input, noncontiguous transfers, and parallel interactions. The following is the sequential specialization needed here: $M = \widetilde{O}(B)$ and $I = O(R)$. Input-dependent initialization can be handled by storing differences from the initial blocks. We reprove this case for completeness, using their read/write circuit organization [SW26, Section 3].

Lemma 4 (Block visits)**.** *The controller after R visits, each touching constantly many B-bit blocks, is computable in $\widetilde{O}(B + R)$ space.*

Proof. Let Z_t be the controller after visit t, and let $C_{t,i}$ be block i at that time. These are circuit values, not stored configurations. Pad with dummy blocks so that the selector returns exactly a distinct indices

$$(i_{t,1}, \ldots, i_{t,a}) = \mathsf{Select}(Z_t).$$

Construct one layer per visit, in three stages.

Read. For each slot j, obtain $D_{t,j} = C_{t,i_{t,j}}$. This is an addressed-read gate: put Z_t publicly on the board and give $C_{t,i}$ to party i. The selector identifies the output party, so communication is zero.

Compute. Apply the local visit routine to $Z_t, D_{t,1}, \dots, D_{t,a}$. Package the selected indices, outgoing controller, and updated blocks as

$$V_t = (i_{t,1}, \dots, i_{t,a}, Z_{t+1}, U_{t,1}, \dots, U_{t,a}).$$

This is an ordinary operation with total input and output length $O(B + c)$.

Write. For every block i, set

$$C_{t+1,i} = \begin{cases} U_{t,j}, & i = i_{t,j} \text{ for some } j, \\ C_{t,i}, & \text{otherwise.} \end{cases}$$

Put V_t publicly on the board and give $C_{t,i}$ to the sole party. This gate also communicates nothing. Extract Z_{t+1} from V_t.

Induction on t proves correctness. The circuit has $O(Rm)$ gates and input occurrences, width $O(B + c) = \widetilde{O}(B)$, depth $O(R)$, and $Q = 0$. Time, type, block, and slot indices determine every gate and its wiring in $\operatorname{polylog} T$ space. Theorem 1 gives $\widetilde{O}(B + R)$. □

Alternatively, implement each addressed read by a balanced binary selection tree. It adds only $O(\log m)$ depth per visit, so Corollary 2 gives the same bound. Thus this specialization uses only ordinary Cook–Mertz evaluation.

5 Short conversations between blocks

The dependence on R improves when the block owners can execute a visit with little communication. The special case of one block is particularly simple: its owner simulates the visit locally and broadcasts only the outgoing controller.

Lemma 5 (Communicating visits)**.** *Suppose each visit has a deterministic protocol among the selected block owners that writes at most κ bits, including the outgoing controller. The incoming controller is public, and every owner can compute its updated block from its original block and the conversation. Assume $c \le \kappa \le \widetilde{O}(B)$ and local computations use $\widetilde{O}(B)$ space.*

Then the controller after R visits is computable in

$$\boxed{\widetilde{O}\big(B + (R\kappa)^{2/3}\big)}$$

space.

All protocol rules and update routines are total on arbitrary Boolean data. Pad each visit record to κ bits, with a fixed field for the outgoing controller. The inputs are the blocks at the beginning of the visit; owners may reconstruct their local state from those inputs and the public conversation whenever a message is needed.

5.1 Replay, epochs, and rounds

Proof of Lemma 5. Given a sequence of complete visit records and the starting contents of block i, its owner can replay the updates to that block. The initial controller and completed records determine all selected blocks. At a visit involving block i, apply its local update rule to its current buffer and that visit's record. On the actual history this gives the actual block, by induction. On any other history it is still a total local operation.

Choose epochs of L visits and rounds of k visits, with $k \le L \le R$. Pad to whole rounds and epochs. There are $E = O(R/L)$ epochs and $J = O(L/k)$ rounds in each. Define

$$A_{e,i} = \text{block } i \text{ at the beginning of epoch } e,$$
$$H_{e,j} = \text{the incoming controller and the first } jk \text{ visit records of epoch } e.$$

Every block and every complete epoch history fits in width

$$w = \widetilde{O}(B + L\kappa). \tag{10}$$

The initial blocks and controller are sources. At epoch boundaries use

$$A_{e+1,i} = \mathsf{Replay}_i(H_{e,J}, A_{e,i}), \qquad H_{e+1,0} = \mathsf{Last}(H_{e,J}). \tag{11}$$

Both are zero-communication gates: the history is public and a block owner computes its own update.

A round gate. To compute $H_{e,j+1}$, put $H_{e,j}$ on the initial blackboard and give $A_{e,i}$ privately to party i. Execute the next k visit protocols. Before participating in a visit, an owner replays the completed epoch records against its epoch-start block. This reconstructs the block at the beginning of the current visit. It can then generate its messages from that block and the current conversation. The output is the enlarged history, and the communication is at most

$$q = k\kappa \le w.$$

The history and one private block suffice for each local computation, using $\widetilde{O}(w)$ space. The same rules define total gates even on inconsistent histories. Induction through visits, rounds, and epochs proves that the circuit has its stated values.

One charged dependency per epoch. A dependency path follows at most J successive history edges in epoch e. These are public edges. If it enters a private block $A_{e,i}$, it pays at most $k\kappa$; that block's dependencies are in epoch $e-1$, by (11). It cannot pay for a second round in epoch e.

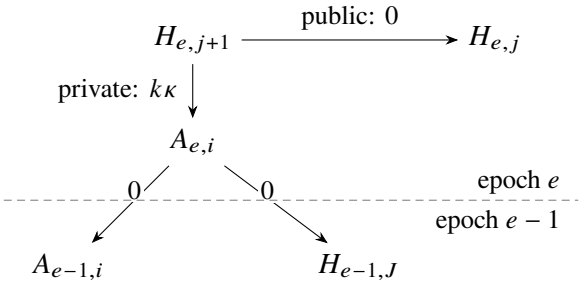


Figure 1: Dependencies point toward inputs. Labels record communication charges. The other route to the preceding epoch is the zero-communication edge $H_{e,0} \to H_{e-1,J}$.

Consequently,

$$h = O(EJ) = O(R/k), \qquad Q = O(Ek\kappa) = O(Rk\kappa/L). \tag{12}$$

The gate names consist of a type and the indices (e,i) or (e,j). Their dependencies are recovered by arithmetic on these indices. There are polynomially many gates and input occurrences in T; the circuit is never materialized.

Theorem 1 and (10)–(12) give

$$\text{space} = \widetilde{O}\left(B + L\kappa + \frac{R}{k} + \frac{Rk\kappa}{L}\right). \tag{13}$$

If $R \ge \kappa^2$, choose

$$k = (R/\kappa^2)^{1/3}, \qquad L = (R^2/\kappa)^{1/3}.$$

Then $L\kappa = R/k = Rk\kappa/L = (R\kappa)^{2/3}$. If $R < \kappa^2$, Lemma 4 gives $\widetilde{O}(B+R)$, which is at most the claimed bound. The final history contains the desired controller. □

Corollary 6 (One block per visit)**.** *The controller after R visits, each touching one B-bit block, is computable in $\widetilde{O}(B + R^{2/3})$ space.*

Proof. The selected block owner runs the visit and broadcasts its $c = O(\log T)$-bit outgoing controller. Its updated block is locally computable. Apply Lemma 5 with $\kappa = c$. □

Ignoring logarithmic factors, one can take $k = R^{1/3}$ and $L = k^2$. More generally this choice of epoch shape gives the useful bound

$$\widetilde{O}\left(B + k^2 + \frac{R}{k}\right) \tag{14}$$

for one-block visits. The exponent $2/3$ balances the size of a complete epoch history against the depth and communication accumulated along a path.

6 Time in low space

6.1 Geometric blocking

Fix a side length b. Partition every d-dimensional tape into cubes

$$C_{\mathbf{v}}^{(s)} = \prod_{j=1}^{d} [s + v_j b,\ s + (v_j + 1)b) \cap \mathbb{Z}^d, \qquad s \in \{0, \dots, b-1\}.$$

Use the same shift s in all coordinates and on all tapes. A nearest-neighbor move crosses a cube boundary for exactly one shift. For a fixed number a of writable heads, summing crossings over all shifts gives at most aT. Some shift therefore has $O(T/b)$ crossings.

A joint visit ends when any head leaves its current cube, or when the machine halts or reaches the cap. The departure step reads and writes the old cubes before moving. For the good shift,

$$B = \Theta(b^d), \qquad R = O(T/b), \tag{15}$$

and a visit touches at most a blocks. Its controller stores the state, head coordinates, elapsed time, and terminal flags, in $O(\log T)$ bits. A local visit uses $\widetilde{O}(B)$ space.

Try every shift, reusing the workspace. Compute the controller after the allotted $O(T/b)$ visits and discard a trial that has not reached halt or timeout. At least one finishes, and all finished trials return the same machine outcome.

Local access. Clock each visit by the remaining time budget. A valid controller at elapsed time t has head coordinates of magnitude at most t; after at most $T - t$ further steps, every coordinate is still within $[-T, T]$. There are polynomially many relevant cubes. Generate their initial contents from the fixed input encoding by indexed rescanning, filling exterior cells with a fixed symbol. Invalid encodings receive fixed defaults and terminal visits are identities. These conventions make every local routine total, including on speculative inputs.

6.2 A fixed number of ordinary tapes

Theorem 7 (Multitape time)**.** *A T-step computation on a fixed number of one-dimensional writable tapes is simulable in $\widetilde{O}(T^{1/2})$ space.*

Proof. Use (15) with $d = 1$ and apply Lemma 4. The space is

$$\widetilde{O}\left(b + \frac{T}{b}\right).$$

Take $b = T^{1/2}$. □

This recovers Williams's exponent [Wil25]. The circuit reduction is the sequential external-memory specialization proved in Section 4; it does not use short conversations.

6.3 One tape, in any fixed dimension

Theorem 8 (One writable tape)**.** *For fixed d, a T-step computation on one d-dimensional writable tape is simulable in*

$$\boxed{\widetilde{O}\big(T^{2d/(3d+2)}\big)}$$

space. In particular, one ordinary tape takes $\widetilde{O}(T^{2/5})$ space.

Proof. A visit now touches one cube. Corollary 6 and (15) give

$$\widetilde{O}\left(b^d + \left(\frac{T}{b}\right)^{2/3}\right). \tag{16}$$

Balance $b^d = (T/b)^{2/3}$, or $b^{3d+2} = T^2$. The choice $b = T^{2/(3d+2)}$ proves the theorem. □

The exponents for dimensions $1, 2, 3$ are $2/5, 1/2, 6/11$. For one ordinary tape, the change from the block-visit lemma to the short-conversation lemma is exactly

$$\underbrace{b + T/b}_{\text{ordinary block simulation}} \qquad \longrightarrow \qquad \underbrace{b + (T/b)^{2/3}}_{\text{short histories}}.$$

At the optimum, suppressing logarithms,

$$B = w = T^{2/5}, \quad R = T^{3/5}, \quad k = E = T^{1/5}, \quad h = Q = T^{2/5}.$$

The word storage, ordinary depth, and path communication have the same scale. The history dependencies do not carry a conversation; a block dependency carries one round and leaves the current epoch.

6.4 Random-access memory

Fix a RAM with a constant number of processor registers, $\ell = O(\log T)$-bit words and addresses, and constantly many memory accesses per instruction. Its word operations are total and computable in $\operatorname{polylog} T$ space. Word length and arithmetic conventions are fixed for the simulated computation. Initial memory words are computable from the input and an address in $\operatorname{polylog} T$ space; uninitialized words are zero.

Theorem 9 (RAM time)**.** *A T-instruction computation of this RAM is simulable in $\widetilde{O}(T^{2/3})$ space.*

Proof. Serialize each instruction into constantly many microsteps, each accessing one memory word. Keep fetched operands, pending results, intermediate addresses, and the instruction phase in the processor state. An indirect address is first fetched and then used in a later microstep. A purely local step may access an unchanged dummy word.

The controller has $O(\log T)$ bits and determines the next address. Regard each memory word as a block. The address universe has polynomial size, $B = O(\log T)$, and there are $R = O(T)$ one-block visits. Corollary 6 gives $\widetilde{O}(B + R^{2/3}) = \widetilde{O}(T^{2/3})$. □

Here the evaluator's word width comes from the epoch histories, not the RAM word size: $k = E = T^{1/3}$ and $w, h, Q = \widetilde{O}(T^{2/3})$.

6.5 Several higher-dimensional tapes

For a fixed number of d-dimensional tapes, geometric blocking and Lemma 4 give

$$\widetilde{O}\left(b^d + \frac{T}{b}\right) = \widetilde{O}\big(T^{d/(d+1)}\big)$$

at $b = T^{1/(d+1)}$. Alternatively, serialize the constantly many tape-cell accesses in a machine step, retaining the head coordinates and read symbols in a logarithmic-size controller. This is a bounded-word RAM computation of time $O(T)$. Thus

$$\boxed{\text{fixed multitape dimension } d: \qquad \widetilde{O}\big(T^{\min\{d/(d+1),\,2/3\}}\big).} \tag{17}$$

More generally, Lemma 5 applies whenever joint visits have short protocols: if a joint cube visit admits communication $\kappa(b)$, the space is also bounded by

$$\widetilde{O}\left(b^d + \left(\frac{T\kappa(b)}{b}\right)^{2/3}\right).$$

No such small-communication condition follows from geometry alone. A joint visit can exchange information repeatedly between independently held blocks even when neither head crosses a boundary.

For a halting computation whose running time is at least the input length, doubling the cap and restarting gives all the same bounds without knowing that running time. The original input encoding and RAM arithmetic conventions remain fixed across trials.

References

[CM24] James Cook and Ian Mertz. *Tree evaluation is in space $O(\log n \cdot \log\log n)$*. STOC 2024, pp. 1268–1278. doi:10.1145/3618260.3649664. ECCC preprint TR23-174.

[OAI26] OpenAI. *Simulating One-Tape Time in Two-Fifths-Power Space*. Manuscript, September 25, 2026.

[SW26] Danil Sibgatullin and Ryan Williams. *Space-Efficient Simulations Beyond Multitape Turing Machines*. ECCC TR26-223, 2026. Full version of the FOCS 2026 paper.

[Wil25] R. Ryan Williams. *Simulating time with square-root space*. STOC 2025, pp. 13–23. arXiv:2502.17779.